📄 PART 1 — ARCHITECTURE & SETUP

🔹 Prompt 1

I am building an AI-powered pipeline failure monitoring system in Snowflake.

Before writing any SQL:

1. Identify all components required
2. Define schemas and their purpose
3. Define data flow from logs → AI → insights
4. Suggest Snowflake features to use (tasks, views, Cortex, etc.)
5. Highlight potential pitfalls

Constraints:

- Must use Snowflake Cortex COMPLETE()
- Must be fully automated using TASKS
- Should capture QUERY, TASK, and SYSTEM failures
- Must be near real-time (1–2 min latency)

Do NOT write SQL yet.
Return structured architecture only.

🔹 Prompt 2

Now create Snowflake setup based on approved architecture.

Context:
This system will monitor failures across pipelines and analyze them using AI.

Create:

1. Database: OPS_AI_MONITOR
2. Schemas:
    - EVENT_LOGS (raw failures)
    - AI_ENGINE (AI outputs)
    - METADATA (reference tables)
3. Warehouse:
    - Name: OPS_MONITOR_WH
    - Size: XSMALL
    - Auto suspend/resume enabled

Constraints:

- Use best naming conventions
- Fully qualified SQL
- Do NOT add unnecessary objects

Return ONLY executable SQL.

🔹 Prompt 3

Create a unified view that captures failures from multiple Snowflake metadata sources.

Sources:

1. INFORMATION_SCHEMA.QUERY_HISTORY
2. INFORMATION_SCHEMA.TASK_HISTORY
3. SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY

Requirements:

- Only FAILED queries/tasks

- Include:
  event_id, source_type, error_message, query_text,
  user_name, warehouse, database, schema, start_time

Constraints:

- Use UNION ALL
- Handle NULLs properly
- Limit recent data (last 5–10 mins for real-time)

Output:
CREATE VIEW SQL only.

📄 PART 2 — DATA PIPELINE + AI CORE

🔹 Prompt 4

Create a table to store all failure events.

Table: OPS_AI_MONITOR.EVENT_LOGS.FAILURE_EVENTS

Columns:

- event_id
- source_type
- error_message
- query_text
- user_name
- warehouse_name
- database_name
- schema_name
- start_time
- created_at (default current timestamp)

Constraints:

- Optimized for incremental ingestion
- Use appropriate data types

Return SQL only.

🔹 Prompt 5

Create a Snowflake TASK that runs every 1 minute.

Function:

- Merge new records from failure view into failure table
- Avoid duplicates using event_id

Constraints:

- Use MERGE
- Warehouse: OPS_MONITOR_WH
- Efficient incremental logic

Return:

1. CREATE TASK
2. MERGE logic
3. Resume statement

🔹 Prompt 6

Create a table to store AI analysis results.

Table: OPS_AI_MONITOR.AI_ENGINE.ERROR_ANALYSIS

Columns:

- event_id
- root_cause
- severity
- suggested_fix
- fix_sql
- confidence_score
- raw_response
- status (SUCCESS / FAILED)
- created_at

Constraints:

- Designed for Cortex JSON parsing
- Must support retries

Return SQL only.

🔹 Prompt 7

Generate Snowflake SQL that uses SNOWFLAKE.CORTEX.COMPLETE()

Context:
We are analyzing pipeline failures.

Input:

- error_message
- query_text

Output JSON format:
{
"root_cause": "",
"severity": "LOW|MEDIUM|HIGH",
"suggested_fix": "",
"fix_sql": "",

"confidence_score": 0.0
}

Rules:

- Always return valid JSON
- fix_sql must be executable
- Handle cases:
    - missing table
    - permission issues
    - syntax errors
    - division by zero

Constraints:

- Strip markdown formatting
- Use TRY_PARSE_JSON

Return ONLY SQL snippet.

🔹 Prompt 8

Create a Snowflake TASK that runs AFTER failure ingestion task.

Function:

- Pick latest unprocessed failures
- Call Cortex COMPLETE()
- Parse JSON
- Insert into AI table

Constraints:

- Limit to 10 records per run (cost control)
- Skip already processed events
- Handle NULL AI response

Return:

1. CREATE TASK
2. MERGE logic
3. Resume order instructions

📄 PART 3 — INSIGHTS + KB + ADVANCED FEATURES

🔹 Prompt 9

Create a final view combining:

- failure logs
- AI analysis

View: OPS_AI_MONITOR.AI_ENGINE.V_FAILURE_INSIGHTS

Output columns:

- event_id
- error_message
- query_text
- start_time
- root_cause
- severity
- suggested_fix
- fix_sql
- confidence_score
- status

Constraints:

- LEFT JOIN
- Optimized for dashboard usage

Return SQL only.

🔹 Prompt 10

Create an enriched view that classifies errors.

Add:

1. pipeline_type:
    - TASK, STORED_PROC, STREAM, DYNAMIC_TABLE, SQL
2. error_category:
    - MISSING_OBJECT
    - PERMISSION
    - SYNTAX
    - RUNTIME
    - OTHER

Use:

- QUERY_TEXT patterns
- ERROR_MESSAGE patterns

Return SQL for view.

🔹 Prompt 11 — Knowledge Base

I am enhancing an AI-powered Snowflake pipeline failure monitoring system.

Context:
We already capture failure logs and analyze them using Snowflake Cortex.

Now we want to improve AI accuracy using a knowledge base of common errors.

Task:
Create a reference table to store error patterns and recommended fixes.

Table:
OPS_AI_MONITOR.METADATA.ERROR_KB

Columns:

- pattern
- root_cause
- recommended_fix
- fix_sql

Insert sample rows:

- "does not exist"
- "object not found"
- "permission denied"
- "insufficient privileges"
- "division by zero"
- "warehouse is suspended"

Constraints:

- Use meaningful root cause and fix descriptions
- Include executable SQL in fix_sql
- Keep it simple and extensible

Output:
CREATE TABLE + INSERT SQL.

🔹 Prompt 12 — AI Queue

Create a view to identify pending failures for AI processing.

View:
OPS_AI_MONITOR.AI_ENGINE.V_AI_QUEUE

Logic:

- Select failures not yet analyzed OR failed previously
- Limit to recent records
- Include only required columns

Constraints:

- Avoid duplicates
- Limit to 20 records
- Optimize for task usage

Output:
CREATE VIEW SQL only.

🔹 Prompt 13 — KB Enhancement

Modify the AI processing logic to use knowledge base hints.

Context:
We have a table OPS_AI_MONITOR.METADATA.ERROR_KB.

Task:
Enhance Cortex prompt so that:

1. Match error_message with KB.pattern
2. Pass best matching KB.recommended_fix as a hint into prompt

Constraints:

- Keep JSON output format unchanged
- Ensure valid JSON parsing

- Must still work even if no KB match

Output:
Return ONLY updated SQL logic.

🔹 Prompt 14 — Documentation

Generate complete documentation for this system.

System Name:
AI Pipeline Failure Investigator

Include:

1. Architecture overview
2. Data flow
3. Snowflake objects
4. AI logic
5. Automation
6. Testing
7. Business value

Output:
Clean documentation text only.

🔹 Prompt 15 — Metrics

I want to add observability metrics to my pipeline monitoring system.

Create a metrics view:
OPS_AI_MONITOR.AI_ENGINE.V_FAILURE_METRICS

Metrics:

- total_failures
- query_failures
- task_failures
- dynamic_table_failures
- snowpipe_failures
- internal_errors
- high_severity_count
- success_count

Output:
CREATE VIEW SQL only.

🔹 Prompt 16 — SLA

I want to detect SLA breaches.

Expected runtime = 5 minutes

Create view:
OPS_AI_MONITOR.AI_ENGINE.V_SLA_BREACH

Output:
CREATE VIEW SQL only.

🔹 Prompt 17 — Custom Logs

I want to support external pipeline logs (Fivetran, DBT, custom ETL).

Create table:
OPS_AI_MONITOR.EVENT_LOGS.CUSTOM_LOGS

Add dummy data.

Output:
CREATE TABLE + INSERT SQL.

🔹 Prompt 18 — Alerts

Create a critical alerts detection system.

Conditions:

- HIGH severity
- SLA breach
- repeated failures

Output:
CREATE VIEW SQL only.

🔹 Prompt 19 — Auto Fix

Create automatic fix execution system.

- Read FIX_SQL
- Execute automatically

Conditions:

- Only LOW/MEDIUM severity
- Skip risky queries

Output:
CREATE TASK SQL.

Prompt 20 — Create Streamlit Dashboard

Create Streamlit Dashboard